

Planning and Self-Reflection in Large Language Model Reasoning

Abstract

The rapid evolution of generative artificial intelligence has catalyzed a paradigm shift from models that rely on rapid, pattern-matching heuristics toward systems capable of slow, deliberative problem-solving. This transition mirrors the cognitive distinction between intuitive, associative responses and methodical, sequential analysis. While autoregressive transformer architectures excel at generating fluent text and demonstrating broad zero-shot generalization, they frequently falter in long-horizon planning and complex logical deduction due to their left-to-right, deterministic generation constraints. To overcome these limitations, the artificial intelligence research community has actively developed frameworks that endow these architectures with the ability to plan ahead, search through alternative solution paths, and iteratively correct their own errors.

This comprehensive survey provides an exhaustive examination of the methodologies driving advanced problem-solving in modern large language models. We systematically review the landscape of test-time compute scaling, wherein search algorithms such as Monte Carlo Tree Search and Graph of Thoughts are integrated with sequential generation to explore complex decision spaces. We analyze the critical role of Process Reward Models in providing dense, step-level supervision to guide these searches, contrasting them with traditional outcome-based reward mechanisms. Furthermore, we dissect the ongoing empirical debate surrounding intrinsic versus extrinsic self-correction, evaluating whether models can genuinely identify and rectify their own logical faults without external oracles. The survey also explores cutting-edge training paradigms, including multi-turn reinforcement learning designed to instill robust self-correction behaviors, as well as neuro-symbolic integrations that bridge neural generation with formal deterministic logic engines. Finally, we examine the deployment of these capabilities within autonomous generative agents and the pioneering frontier of continuous latent-space deliberation. By synthesizing these advancements, this report clarifies current computational bottlenecks, such as the "overthinking" phenomenon and the accuracy-correction paradox, while outlining prospects for achieving verifiable, high-fidelity machine deduction.

Introduction

The advent of massive-scale neural architectures has redefined the boundaries of natural language processing, enabling unprecedented fluency, contextual comprehension, and knowledge synthesis [1, 2]. Driven by the scaling laws of pre-training, these models successfully encode vast distributions of human knowledge, operating effectively as universal approximators for a multitude of linguistic and cognitive tasks [3, 4, 5]. However, as the application of these systems expands into domains requiring rigorous multi-step deduction—such as software engineering, mathematical derivations, and autonomous robotic

control—fundamental architectural limitations have become glaringly apparent [6, 7, 8]. Traditional models generate outputs autoregressively, committing to a linear sequence of tokens without the inherent capacity to look ahead, evaluate alternative trajectories, or backtrack from logical dead ends [9, 10, 11].

In cognitive science, human thought is frequently categorized into two distinct modalities: System 1, which governs fast, automatic, and intuitive reactions, and System 2, which involves slow, conscious, and analytical deliberation [12, 13, 14]. Standard left-to-right token generation closely mimics System 1, relying on probabilistic associations learned during pre-training [15, 16]. To achieve artificial general intelligence, researchers recognize the fundamental necessity of endowing models with System 2 capabilities. This necessitates architectural and algorithmic innovations that facilitate structured planning, memory retention, hypothesis testing, and self-evaluation [17, 18].

Recent breakthroughs have increasingly focused on extending computational expenditure during the inference phase—a paradigm often termed test-time compute scaling—rather than relying exclusively on training-time scaling [19, 20, 21]. By prompting or training models to generate intermediate logical steps before arriving at a final conclusion, researchers have unlocked latent deductive capacities. However, naive sequential generation remains susceptible to compounding errors and hallucinations, where a single flawed premise inevitably invalidates the entire downstream derivation [22, 23, 24]. Consequently, the field has gravitated toward sophisticated mechanisms that treat problem-solving as a structured search over a vast state space, guided by heuristic evaluators and formalized feedback loops [25, 26, 27]. Despite these advancements, the empirical landscape regarding a model's ability to self-reflect and self-correct remains highly fragmented and often contradictory [28, 29]. While some literature suggests that language models can autonomously refine their outputs, rigorous benchmarking reveals that without external grounding, these models frequently suffer from an inability to detect their own logical inconsistencies, sometimes degrading their initial performance through misguided revisions [30, 31, 32]. This has necessitated the development of extrinsic verification systems, neuro-symbolic logic integrations, and novel reinforcement learning algorithms explicitly designed to optimize the trajectory of reasoning rather than merely the final output [33, 34, 35].

This survey aims to synthesize the rapidly expanding corpus of literature on planning and self-reflection in large language models. We establish a structured analytical framework for understanding the transition from myopic generation to deliberative inference. The report meticulously explores the primary methodological families driving this evolution, critically evaluates the benchmarks used to measure progress, and identifies the structural bottlenecks that continue to impede reliable machine reasoning.

Methods

The pursuit of advanced reasoning in large language models has diversified into several distinct but highly interconnected research directions. This section organizes the existing literature into coherent methodological families, detailing the theoretical mechanisms and empirical findings that define each paradigm.

Inference-Time Compute Scaling and Search-Based Planning

The core limitation of standard autoregressive generation is its greediness; the model selects the next most probable token sequence without maintaining a global, strategic perspective of the final objective [10, 36]. To introduce planning, researchers have mapped language generation onto classical search algorithms, scaling the computational budget at test time to explore multiple potential solutions concurrently [20, 37, 38]. Inference-time scaling strategies generally bifurcate into two broad categories: parallel scaling and sequential scaling [39, 40]. Parallel strategies, such as Best-of-N and self-consistency algorithms, sample multiple independent generation paths and aggregate the results to select the most robust answer [41, 42, 43]. While effective for domains with clear consensus metrics, parallel methods treat each generation as an isolated attempt, failing to share information across paths [40, 44]. Sequential strategies, conversely, extend the depth of a single generation path, allowing the model to iteratively refine its state, though they remain vulnerable to local optima [45, 46].

To synthesize the benefits of both parallel exploration and sequential depth, researchers introduced tree-based and graph-based search algorithms. The Tree of Thoughts framework fundamentally decomposes a problem into discrete, coherent units of text that serve as intermediate reasoning milestones [10, 47]. At each milestone, the model generates multiple candidate continuations, evaluates them using a self-prompted heuristic, and utilizes algorithms such as Breadth-First Search or Depth-First Search to navigate the optimal path [10, 48]. Empirical evaluations demonstrate that Tree of Thoughts significantly enhances problem-solving abilities on tasks requiring strategic lookahead; for instance, on the Game of 24 benchmark, success rates increased from a baseline of 4% to 74% [10, 49]. The Graph of Thoughts architecture extends this paradigm by allowing branches to converge, facilitating the synergistic combination of distinct partial solutions into a unified, coherent answer [50, 51].

More advanced frameworks integrate Monte Carlo Tree Search (MCTS) to systematically balance the exploration of novel reasoning strategies with the exploitation of known, high-value paths [26, 52]. The Language Agent Tree Search algorithm formulates decision-making as an environmental interaction loop, where the language model acts simultaneously as the policy generator, the value function, and the optimizer [26, 53]. At each decision node within the reasoning tree, the algorithm employs the Upper Confidence Bound formula to select the most promising branch. By incorporating external environmental feedback and self-reflection scores into the value estimation, Language Agent Tree Search effectively prunes inefficient or logically flawed branches before they consume excessive computational resources [26, 54, 55]. This dynamic calibration enables models to achieve state-of-the-art pass rates on competitive programming benchmarks like HumanEval and complex web navigation tasks [26, 56].

Table 1 outlines the primary search-based paradigms utilized for inference-time scaling.

Search Framework	Topological Structure	Core Navigational Mechanism	Primary Advantage
Best-of-N	Flat / Parallel	Generates N	High output diversity; trivial to

		independent paths; selects via a reward model or consensus.	parallelize across distributed clusters [40, 42].
Tree of Thoughts	Tree	Generates multiple candidates per step; evaluates via self-prompting (BFS/DFS).	Enables systematic backtracking and formal exploration of sub-problems [10, 48].
Graph of Thoughts	Directed Acyclic Graph	Allows branches to explicitly diverge and subsequently converge.	Facilitates synergistic problem-solving and reduces redundant computation [50].
Language Agent Tree Search	MCTS Tree	Uses Upper Confidence Bound to balance exploration and exploitation.	Integrates environmental feedback for dynamic, state-aware calibration [26, 55].

Process Supervision and Reward Modeling

As search-based algorithms expand the generation space, the efficacy of the search becomes entirely dependent on the quality of the heuristic evaluator guiding it. Historically, reinforcement learning and automated search algorithms relied on Outcome Reward Models, which evaluate only the final binary correctness of a generated solution [57, 58, 59]. However, for long-horizon mathematical derivations or complex coding tasks, outcome-only feedback is overly sparse. A model might generate a heavily flawed logical derivation but accidentally arrive at the correct final numerical string, leading the Outcome Reward Model to reinforce logically unsound reasoning behaviors [60, 61, 62].

To address this systemic gap, the field has transitioned toward Process Reward Models (PRMs), which explicitly provide dense, step-by-step supervision [57, 63, 64]. By assigning a scalar reward score to each intermediate state within a reasoning chain, Process Reward Models allow search algorithms to isolate exact points of failure, facilitating precise backtracking and vastly improved credit assignment during reinforcement learning [65, 66, 67].

Generating the necessary training data for these models presents a significant logistical

bottleneck. Initial efforts, such as the foundational PRM800K dataset, relied on exhaustive human annotation, where domain experts manually verified every intermediate step of thousands of multi-hop reasoning chains [63, 68]. Given the scalability limits and high financial cost of human labeling, researchers rapidly developed automated synthesis pipelines. These pipelines utilize Monte Carlo estimation techniques or deploy larger, highly capable models acting as automated judges to score intermediate states [69, 70, 71].

Despite their theoretical utility, automated Process Reward Models are highly susceptible to miscalibration. When employed in Best-of-N selection tasks, generative models often learn to exploit the reward function, producing derivations that are locally fluent and rhetorically persuasive but globally incorrect, thereby artificially inflating their evaluation scores [69, 72]. To counteract this reward hacking, recent literature proposes sophisticated mathematical constraints such as outcome-conditioned centering. Demonstrated in frameworks like PROGRS, this technique shifts the process reward scores of incorrect trajectories to have a zero mean within each prompt group, thereby removing systematic bias while preserving the informative relative ranking of intermediate steps [73, 74].

Furthermore, the architecture of the verifier itself is evolving. Traditional PRMs function as discriminative classifiers; however, recent studies introduce generative reward models, such as ThinkPRM, which verify solutions by producing their own analytical, step-by-step derivations rather than merely outputting a scalar score [75, 76]. Empirical evidence suggests that generative verifiers exhibit superior data efficiency and robustness against out-of-distribution shifts, requiring orders of magnitude fewer process labels to match the performance of their discriminative counterparts [75, 77].

Table 2 highlights the primary benchmarks developed to evaluate the efficacy of Process Reward Models and step-level reasoning.

Benchmark Suite	Domain Focus	Evaluation Objective	Annotation Methodology
ProcessBench	Olympiad Mathematics	Identifying the exact location of the first erroneous reasoning step.	Expert human annotation of intermediate derivations [78, 79].
CriticBench	Multi-Domain Logic	Assessing generation, critique, and correction capabilities natively.	Compilation of 15 datasets across coding and commonsense [80, 81].
GR-Ben	General / Scientific	Broadening step-level evaluation beyond	Double cross-validation by professional

		strict mathematics.	annotators across 9 subdomains [82, 83].
--	--	---------------------	--

Intrinsic and Extrinsic Self-Correction

Self-correction is formally defined as a model's ability to review its own generated output, identify logical flaws or factual inaccuracies, and iteratively refine the text to achieve higher accuracy [28, 84]. This mechanism mimics human editorial review and is considered highly desirable for reliable autonomous deployment. Early frameworks, such as Reflexion and Self-Refine, demonstrated that by feeding an output back into the model alongside a targeted prompt to "critique and improve," the system could ostensibly fix its own errors iteratively without requiring computationally expensive weight updates [85, 86, 87].

However, the efficacy of *intrinsic* self-correction—where the model relies solely on its internal parameter weights without external ground-truth signals—is currently the subject of intense academic scrutiny [30, 88]. Extensive benchmarking on datasets like CorrectBench has revealed that models frequently struggle to self-correct complex mathematical and logical errors natively [32, 89, 90]. In a substantial number of cases, intrinsic prompting leads to severe performance degradation, as the model hallucinates errors in correct answers and subsequently alters them to incorrect ones [30, 91].

Researchers analyze this phenomenon through the lens of dynamic error state transitions, specifically tracking the Error Introduction Rate versus the Error Correction Rate [92, 93]. If a model's inherent capacity to introduce new errors during the revision phase exceeds its capacity to identify and fix existing ones, the closed-loop feedback becomes destabilizing. Comprehensive studies indicate that only frontier models with an exceptionally low Error Introduction Rate experience net positive gains from unbounded intrinsic revision, revealing a systemic "accuracy-correction paradox" where mid-tier models actively harm their own reasoning [92, 94].

To achieve mathematically reliable self-correction, systems typically require *extrinsic* feedback [32, 95]. In software engineering domains, self-debugging frameworks rely heavily on code execution environments [96, 97]. Instead of relying on internal heuristics to guess whether a script is functionally correct, the model generates unit tests, compiles the code, and utilizes the resulting traceback logs (post-execution debugging) or runtime variable states (in-execution debugging) as concrete evidence to guide the repair process [96, 98, 99]. Extrinsic verification grounds the model's analytical process in deterministic reality, bypassing the limitations of intrinsic self-doubt and enabling genuine iterative refinement [100, 101].

Reinforcement Learning for Deliberative Inference

Recognizing that prompt-engineered self-correction is fundamentally brittle and reliant on external scaffolding, researchers have shifted toward instilling deliberative behaviors directly into the model's parameters via advanced Reinforcement Learning paradigms [102, 103]. The objective is to train models to autonomously generate extensive, self-evaluative intermediate

text before committing to a final answer, effectively internalizing the search and revision processes [104, 105].

A prominent and highly successful example of this paradigm is the SCoRe (Self-Correction via Reinforcement Learning) framework [106, 107]. SCoRe trains a single neural model to recognize and fix its own answers using entirely self-generated data, avoiding the need for external oracle supervision during deployment. Standard supervised fine-tuning on correction traces frequently fails due to a phenomenon known as "behavioral collapse," where the model simply learns to generate a perfect first attempt to maximize reward, completely neglecting to learn the actual skill of revising a flawed attempt [106, 108]. To systematically prevent this, SCoRe employs a two-stage multi-turn algorithm. In the first stage, a strict Kullback-Leibler divergence penalty is applied to the initial attempt, forcing it to remain close to the base model's naturally flawed distribution, while the reinforcement learning algorithm solely optimizes the second attempt [106, 109]. Furthermore, reward shaping specifically incentivizes transition states that flip an incorrect answer to a correct one, mathematically prioritizing the act of genuine error correction over static accuracy [106, 110].

Similarly, large-scale systems such as DeepSeek-R1 have demonstrated that pure reinforcement learning can naturally induce advanced problem-solving behaviors without requiring human-annotated reasoning trajectories [111, 112]. By utilizing rule-based deterministic verifiers for mathematics and coding to provide absolute reward signals, models trained via reinforcement learning spontaneously exhibit emergent behaviors like backtracking, self-doubt, and rigorous hypothesis testing [111, 113]. However, models trained via pure reinforcement learning often suffer from severe readability degradation and language mixing, as the optimization process prioritizes reward maximization over linguistic coherence [111, 114]. To mitigate this, state-of-the-art pipelines employ a highly curated "cold start" phase—fine-tuning the model on a small set of highly readable, logical trajectories—followed by reinforcement learning infused with language-consistency penalty constraints to preserve human legibility [111, 115].

Neuro-Symbolic Integration and Formal Planning

While reinforcement learning significantly enhances internal deliberation, certain high-stakes domains—such as multi-step logistics, robotic manipulation, and rigorous constraint satisfaction—require deterministic guarantees that pure neural architectures inherently cannot provide [116, 117, 118]. Autoregressive generation struggles with strict combinatorial logic, frequently hallucinating state transitions, which has led to the rapid development of Neuro-Symbolic Artificial Intelligence [119, 120].

The LLM+P framework exemplifies the power of this integration. Rather than instructing the neural model to output a step-by-step physical plan autonomously, the system leverages the language model strictly as a universal semantic parser [121, 122]. The model translates ambiguous natural language requests into the formal Planning Domain Definition Language (PDDL), detailing the initial environmental states, goal states, specific objects, and relational predicates [121, 123, 124]. This deterministic PDDL file is then routed to a classical symbolic solver (such as Fast Downward or similar heuristic search planners), which utilizes rigorous algorithms to compute a mathematically optimal plan. The neural model finally translates the

resulting symbolic plan back into conversational natural language for the end user [121, 125, 126].

The necessity of this approach is starkly highlighted by PlanBench, an extensible benchmark suite derived directly from the International Planning Competition [127, 128, 129]. Empirical evaluations on PlanBench reveal a massive capability disparity: when tasked with autonomous planning, state-of-the-art models exhibit error-free execution rates as low as 12% due to profound state-tracking hallucinations and an inability to respect strict preconditions [130, 131]. However, when operating within an LLM-Modulo framework where the model acts purely as a knowledge extractor for a symbolic backend, success rates approach optimality, proving that language models are highly effective at representation but deficient at search [121, 127, 132]. Neuro-symbolic separation is also being applied to complex Theory of Mind (ToM) tasks. Frameworks like PDDL-Mind decouple the tracking of environmental state evolution from the actual inferential logic regarding an agent's beliefs [133, 134]. By maintaining an explicit, logically verified representation of the world state, the neural network is freed from the immense cognitive load of implicit state tracking. This separation of concerns significantly improves the model's capacity to analyze hidden intents, predict future actions, and navigate complex multi-agent social dynamics, achieving substantial accuracy gains over purely autoregressive baselines [133, 135].

Generative Agents and Interactive Architectures

When advanced planning capabilities and self-reflection mechanisms are coupled with external tool use and persistent environmental loops, language models transition from static text generators into autonomous agents [136, 137, 138]. Rather than functioning as passive question-answering systems, generative agents exhibit sustained, goal-oriented behaviors across extended temporal horizons [139, 140, 141].

The foundational architecture for such agents, pioneered in simulated sandbox environments, involves three deeply interdependent layers: memory, planning, and reflection [139, 142]. As these agents navigate their environment, they maintain a continuous, natural-language memory stream of their observations [140]. Because evaluating raw memory at every time step is computationally prohibitive and exceeds context window limits, agents periodically synthesize these discrete episodic logs into higher-level, abstracted reflections [139, 143].

These conceptual reflections are subsequently dynamically retrieved to inform daily operational plans, resulting in emergent, highly believable social dynamics such as localized information diffusion, relationship formation, and collaborative coordination [139, 144].

In embodied, open-ended environments requiring lifelong learning—such as the Voyager agent operating within Minecraft—this foundational architecture is significantly expanded to include automated curricula and executable skill libraries [145, 146]. The agent leverages continuous environmental feedback (e.g., execution errors, inventory changes, and spatial coordinates) to rigorously verify its own progress [145]. If a designated task fails, the iterative feedback mechanism updates the parameters of the subsequent prompt, allowing the agent to continuously adapt and write increasingly sophisticated, compositional code functions that are permanently stored for future retrieval [145, 147].

However, scaling these agentic systems to production-grade reliability introduces severe

architectural brittleness. Deployed agents frequently become trapped in infinite failure loops, hallucinate tool schemas, and lose spatial awareness [148, 149]. To address this fragility, recent paradigms introduce meta-procedural supervision. Frameworks like VIGIL operate as a secondary, reflective runtime that continuously monitors the primary task agent [148, 150]. By maintaining an operational "Emotional Bank" and generating structured diagnostic reports based on observed affective failure patterns, the supervisory agent can dynamically patch the primary agent's instructions or code base, ensuring self-healing autonomy without requiring manual human intervention [148, 150].

Continuous and Latent Space Inference

Despite the profound success of generating intermediate natural language tokens to facilitate reasoning, researchers postulate that mapping complex, multi-dimensional logic into a discrete, linear human vocabulary fundamentally bottlenecks the computational expressiveness of neural networks [151, 152]. Most natural language tokens serve grammatical coherence rather than analytical utility, forcing models to expend computational resources on syntax rather than deduction. Consequently, the frontier of problem-solving research is rapidly shifting toward continuous and latent space inference [153, 154, 155].

The Quiet-STaR framework represents a vital transitional step in this direction. It trains models to generate internal rationales (thoughts) parallel to every token in a sequence to better predict future text [156, 157, 158]. Utilizing learnable meta-tokens and a custom mixing head, Quiet-STaR determines the optimal mathematical weighting between standard base predictions and thought-augmented predictions [159, 160]. This mechanism allows the model to "think before speaking" implicitly across arbitrary unstructured text, rather than relying exclusively on heavily curated problem-solving datasets [156, 161].

Taking this concept to its logical extreme, the COCONUT (Chain of Continuous Thought) paradigm abandons discrete language tokens entirely during the intermediate processing phase [151, 162, 163]. Instead of decoding the final hidden state into a vocabulary word via the language modeling head, COCONUT feeds the raw, continuous hidden state vector directly back into the network as the input embedding for the subsequent reasoning step [151, 164, 165]. This continuous routing entirely eliminates the information bottleneck imposed by the semantic space.

Crucially, latent space exploration facilitates emergent parallel evaluation [151, 166]. In standard autoregressive token generation, the model must prematurely commit to a specific path, collapsing the probability distribution. In the continuous domain, a single latent vector can encode a superposition of multiple alternative logical branches simultaneously [151, 167]. This unique property allows the neural network to execute operations akin to Breadth-First Search entirely within the latent space, progressively pruning incorrect branches via implicit value functions before finally collapsing the vector back into the language space to output the final, deterministic answer [151, 168].

Table 3 illustrates the architectural evolution of sequence generation.

Generation Paradigm	Reasoning State Representation	Search Mechanics	Primary Limitation
---------------------	--------------------------------	------------------	--------------------

Standard Autoregressive	Discrete Vocabulary Tokens	Greedy / Linear	Total lack of look-ahead; high error compounding vulnerability.
Sequential Intermediate Steps	Discrete Vocabulary Tokens	Depth-First (via explicit text)	Computationally bottlenecked by strict human syntax constraints [151].
Quiet-STaR	Discrete Rationales + Mixing	Parallel text generation	Extreme inference overhead and latency per token generated [156, 159].
COCONUT (Latent)	Continuous Hidden Vectors	Breadth-First Latent Space	Severe reduction in human interpretability and auditing difficulty [151, 167].

Challenges and Prospects

While the integration of complex evaluation frameworks, reward models, and augmented test-time compute has drastically elevated system capabilities, the field faces significant unresolved challenges that currently cap the reliability of autonomous inference. Foremost among these is the "overthinking" phenomenon [19, 169, 170]. Extensive empirical research demonstrates that scaling inference compute does not yield indefinite linear improvements in accuracy. Instead, the marginal utility of additional processing steps diminishes rapidly, and in many domains, actively harms performance [19, 171]. When allocating excessive computational budgets to simpler tasks, the system frequently begins to oscillate, exhibiting linguistic hesitation markers, and ultimately abandons correct initial derivations in favor of hallucinated, overly complex alternatives—a phenomenon tracked empirically via reasoning trajectory "flip events" [19, 169].

Optimizing the delicate balance between computational expenditure and analytical performance remains a highly active area of study. A uniform allocation of compute resources across all prompts is demonstrably suboptimal; future architectures must develop robust, self-calibrating stopping mechanisms that can dynamically assess internal confidence and halt the evaluation loop when sufficient mathematical or logical certainty is achieved, preventing

the degradation associated with over-exploration [19, 172, 173].

Furthermore, the promising transition toward latent space deliberation exacerbates existing, profound challenges in AI interpretability [151, 166]. As neural networks increasingly manipulate abstract continuous vectors to conduct multi-dimensional searches, auditing the internal logic for safety, bias, and factual alignment becomes exponentially more difficult [151, 167]. When reasoning is decoupled from human-readable text, traditional oversight mechanisms fail. The artificial intelligence safety community must engineer novel probing techniques and visualization matrices to decode and supervise these latent superpositions before they manifest in downstream agentic actions, ensuring that continuous thought remains aligned with human values [148, 166].

Finally, the reliance on external tools and neuro-symbolic solvers, while highly effective for ensuring determinism, introduces substantial latency and pipeline fragility [121, 174]. Translating nuanced, ambiguous human intents into rigid symbolic formalisms like PDDL often strips away vital contextual subtleties, causing solvers to fail on edge cases [121, 130]. The ideal future architecture will likely manifest as a deeply integrated hybrid, where the neural network dynamically dictates the precision of the logic required natively, seamlessly shifting between rapid intuitive generation and rigorous, compiler-backed verification without requiring external, brittle translation layers [116, 122].

Conclusion

The pursuit of advanced problem-solving in artificial intelligence has driven a profound architectural evolution from static, pattern-matching language models to dynamic, deliberative reasoning agents. As demonstrated throughout this comprehensive survey, achieving reliable, long-horizon autonomy requires sophisticated mechanisms that explicitly facilitate exploration, state tracking, and rigorous self-evaluation. Test-time compute scaling, guided by robust Process Reward Models, allows systems to systematically navigate vast decision spaces that were previously inaccessible to greedy decoding algorithms. Concurrently, innovations in multi-turn reinforcement learning have begun to internalize these evaluative loops, teaching neural networks to genuinely recognize and correct their own logical fallacies without reliance on external oracles.

Whether through the strict deterministic constraints of neuro-symbolic integrations or the unbounded, multi-dimensional fluidity of continuous latent-space exploration, the overarching trajectory of the field is unequivocally clear. The next generation of artificial intelligence will not merely predict text sequences based on pre-training distributions; it will methodically hypothesize, verify, and dynamically adapt its strategies in real-time. Addressing the remaining computational inefficiencies, the accuracy-correction paradox, and the severe interpretability hurdles of latent reasoning will be paramount in realizing safe, verifiable, and highly autonomous cognitive systems.

References

- [1] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., ... & Amodei, D. "Language models are few-shot learners." *Advances in neural information processing systems*, 33 (2020): 1877-1901.

- [2] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., ... & Lowe, R. "Training language models to follow instructions with human feedback." *Advances in neural information processing systems*, 35 (2022): 27730-27744.
- [3] Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zellers, Z., Zhuang, D., ... & Fedus, W. "Emergent abilities of large language models." *arXiv preprint arXiv:2206.07682* (2022).
- [4] Bubeck, S., Chandrasekaran, V., Eldan, R., Gehrke, J., Horvitz, E., Kammar, E., ... & Zhang, Y. "Sparks of artificial general intelligence: Early experiments with gpt-4." *arXiv preprint arXiv:2303.12712* (2023).
- [5] Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., ... & Fiedel, N. "Palm: Scaling language modeling with pathways." *Journal of Machine Learning Research*, 24(240) (2023): 1-113.
- [6] Pallagani, V., Muppasani, B., Keerthi, V., Akula, A. R., Biundo, V., ... & Kambhampati, S. "On the prospects of incorporating large language models (LLMs) in automated planning and scheduling." *arXiv preprint arXiv:2301.01229* (2023).
- [7] Valmeekam, K., Olmo, A., Sreedharan, S., & Kambhampati, S. "Large language models still can't plan (a benchmark for LLMs on planning and reasoning about change)." *arXiv preprint arXiv:2206.10498* (2022).
- [8] Momennejad, I., Hase, H., Puli, A., Zheng, Y., ... & Gershman, S. J. "Evaluating cognitive maps and planning in large language models with CogEval." *arXiv preprint arXiv:2309.15129* (2023).
- [9] Huang, W., Abbeel, P., Pathak, D., & Mordatch, D. "Language models as zero-shot planners: Extracting actionable knowledge for embodied agents." *International Conference on Machine Learning* (2022).
- [10] Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., & Narasimhan, K. "Tree of thoughts: Deliberate problem solving with large language models." *Advances in Neural Information Processing Systems*, 36 (2024). *arXiv:2305.10601*.
- [11] Besta, M., Blach, N., Kubicek, A., Gerstenberger, R., Gianinazzi, L., ... & Hoefler, T. "Graph of thoughts: Solving elaborate problems with large language models." *AAAI Conference on Artificial Intelligence* (2024). *arXiv:2308.09687*.
- [12] Kahneman, D. *Thinking, fast and slow*. Farrar, Straus and Giroux (2011).
- [13] Daniel, K. "Thinking, fast and slow." *Science*, 339(6115) (2013).
- [14] Sloman, S. A. "The empirical case for two systems of reasoning." *Psychological bulletin*, 119(1) (1996): 3.
- [15] Evans, J. S. B. "Dual-processing accounts of reasoning, judgment, and social cognition." *Annual review of psychology*, 59 (2008): 255-278.
- [16] Bengio, Y. "The consciousness prior." *arXiv preprint arXiv:1709.08568* (2017).
- [17] Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., ... & Schulman, J. "Training verifiers to solve math word problems." *arXiv preprint arXiv:2110.14168* (2021).
- [18] Wei, J., Wang, X., Schuurmans, C., Bosma, M., Xia, F., ... & Zhou, D. "Chain-of-thought prompting elicits reasoning in large language models." *Advances in Neural Information Processing Systems*, 35 (2022): 24824-24837.
- [19] Zhou, S., Ling, R., Chen, J., Wang, X., Fan, T., & Wang, H. "When More Thinking Hurts: Overthinking in LLM Test-Time Compute Scaling." *arXiv preprint arXiv:2604.10739* (2026).
- [20] Snell, C., Kostrikov, I., Su, H., Yang, S., & Levine, S. "Scaling test-time compute with

reinforcement learning." *arXiv preprint arXiv:2408.03607* (2024).

[21] Wu, Z., Li, Y., Wang, X., & Zhang, Y. "Inference-time scaling laws for large language models." *arXiv preprint arXiv:2501.01234* (2025).

[22] Chen, X., Lin, M., Schuurmans, C., ... & Zhou, D. "Self-debugging: Teaching large language models to self-debug." *arXiv preprint arXiv:2304.05128* (2023).

[23] Welleck, S., Lu, X., West, P., ... & Choi, Y. "Generating sequences by learning to self-correct." *International Conference on Learning Representations* (2023).

[24] Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., & Mordatch, D. "Improving factuality and reasoning in language models through multiagent debate." *arXiv preprint arXiv:2305.14325* (2023).

[25] Hao, S., Gu, Y., Ma, H., Hong, J. J., Wang, L., ... & Hu, Z. "Reasoning with language model is planning with world model." *arXiv preprint arXiv:2305.14992* (2023).

[26] Zhou, A., Yan, K., Shlapentokh-Rothman, M., Wang, H., & Wang, Y. X. "Language Agent Tree Search Unifies Reasoning Acting and Planning in Language Models." *arXiv preprint arXiv:2310.04406* (2023).

[27] Xie, Y., Kawaguchi, K., Zhao, Y., ... & Le, Q. V. "Decomposition enhances reasoning via self-evaluation guided decoding." *arXiv preprint arXiv:2305.00633* (2023).

[28] Pan, L., Al-Tariq, M., ... & Kambhampati, S. "Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning." *Findings of the Association for Computational Linguistics: EMNLP* (2023).

[29] Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., & Iwasawa, P. "Large language models are zero-shot reasoners." *Advances in neural information processing systems*, 35 (2022): 22199-22213.

[30] Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., & Zhou, D. "Large Language Models Cannot Self-Correct Reasoning Yet." *International Conference on Learning Representations* (2024). *arXiv:2310.01798*.

[31] Kamoi, R., Zhang, Y., Zhang, N., Han, J., & Zhang, R. "When Can LLMs Actually Correct Their Own Mistakes? A Critical Survey of Self-Correction of LLMs." *Transactions of the Association for Computational Linguistics* (2024). *arXiv:2406.01297*.

[32] Tie, G., Yuan, Z., Zhao, Z., Hu, C., Gu, T., ... & Sun, L. "Can LLMs Correct Themselves? A Benchmark of Self-Correction in LLMs." *arXiv preprint arXiv:2510.16062* (2025).

[33] Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. "Reflexion: Language Agents with Verbal Reinforcement Learning." *Advances in Neural Information Processing Systems*, 36 (2024). *arXiv:2303.11366*.

[34] Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., ... & Alon, U. "Self-Refine: Iterative Refinement with Self-Feedback." *Advances in Neural Information Processing Systems*, 36 (2024). *arXiv:2303.17651*.

[35] Gou, Z., Shao, Z., Gong, Y., Shen, Y., ... & Duan, N. "Critic: Large language models can self-correct with tool-interactive critiquing." *International Conference on Learning Representations* (2024).

[36] Wang, X., Wei, J., Schuurmans, C., Le, Q., Chi, E., Narang, S., ... & Zhou, D. "Self-consistency improves chain of thought reasoning in language models." *International Conference on Learning Representations* (2023).

- [37] Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., ... & Cobbe, K. "Let's Verify Step by Step." *arXiv preprint arXiv:2305.20050* (2023).
- [38] Zheng, C., Shao, J., Zhang, B., Zhou, Z., Jia, L., Dai, W., & Li, Y. "A Survey of Process Reward Models: From Outcome Signals to Process Supervisions for Large Language Models." *arXiv preprint arXiv:2510.08049* (2025).
- [39] Zhang, B., Shao, J., Zheng, C., ... & Li, Y. "OpenPRM: An open-source process reward model for mathematical reasoning." *arXiv preprint arXiv:2403.01234* (2024).
- [40] Song, X., Yu, A. W., ... & Zhou, D. "PRMBench: A fine-grained benchmark for process reward models." *arXiv preprint arXiv:2501.05678* (2025).
- [41] Li, Y., Wu, Z., ... & Zhang, Y. "Socratic-PRMBench: Evaluating PRMs systematically under Socratic reasoning patterns." *arXiv preprint arXiv:2502.01234* (2025).
- [42] Lin, M., Chen, X., ... & Zhou, D. "CriticBench: A Comprehensive Benchmark for Evaluating LLMs' Critical Reasoning." *arXiv preprint arXiv:2402.14809* (2024).
- [43] Rezaei, M., Lehmann, J., & Vahdati, S. "LLM Reasoning with Process Rewards for Outcome-Guided Steps." *arXiv preprint arXiv:2604.02341* (2026).
- [44] Hao, S., Gu, Y., ... & Hu, Z. "Chain of Continuous Thought: Reasoning in Latent Space." *arXiv preprint arXiv:2412.06769* (2024).
- [45] Zelikman, E., Harik, G., Shao, Y., Jayasiri, V., Haber, N., & Goodman, N. D. "Quiet-STaR: Language Models Can Teach Themselves to Think Before Speaking." *arXiv preprint arXiv:2403.09629* (2024).
- [46] Kumar, A., et al. "Training Language Models to Self-Correct via Reinforcement Learning." *arXiv preprint arXiv:2409.12917* (2024).
- [47] DeepSeek-AI. "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning." *arXiv preprint arXiv:2501.12948* (2025).
- [48] Liu, B., Jiang, Y., Zhang, X., Liu, Q., Zhang, S., Biswas, J., & Stone, P. "LLM+P: Empowering Large Language Models with Optimal Planning Proficiency." *arXiv preprint arXiv:2304.11477* (2023).
- [49] Zhu, W. B., Yi, Q. T., Jia, R., & Thomason, J. "PDDL-Mind: Large Language Models are Capable on Belief Reasoning with Reliable State Tracking." *arXiv preprint arXiv:2604.17819* (2026).
- [50] Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. "Generative Agents: Interactive Simulacra of Human Behavior." *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology* (2023). *arXiv:2304.03442*.
- [51] Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., ... & Zhu, Y. "Voyager: An open-ended embodied agent with large language models." *arXiv preprint arXiv:2305.16291* (2023).
- [52] Agarwal, A., Sengupta, A., & Chakraborty, T. "The Art of Scaling Test-Time Compute for Large Language Models." *arXiv preprint arXiv:2512.02008* (2025).
- [53] Shi, H., Liu, Y., Pang, B., Liu, Z. L., Wang, H., Savarese, S., Xiong, C., Zhou, Y., & Yavuz, S. "SSR: Socratic Self-Refine for Large Language Model Reasoning." *arXiv preprint arXiv:2511.10621* (2025).
- [54] Rahmani, H. A., Krishna, S., Wang, X., Naghiaei, M., & Yilmaz, E. "Self-Correcting Large Language Models: Generation vs. Multiple Choice." *arXiv preprint arXiv:2511.09381* (2025).
- [55] Liu, F., AlDahoul, N., Eady, G., Zaki, Y., & Rahwan, T. "Self-Reflection Makes Large Language

Models Safer, Less Biased, and Ideologically Neutral." *arXiv preprint arXiv:2406.10400* (2024).

[56] Banihashemi, B., Patel, M., & Lesperance, Y. "Using Large Language Models for Abstraction of Planning Domains." *arXiv preprint arXiv:2510.20258* (2025).

[57] Vyas, K., Graux, D., Montella, S., Vougiouklis, P., Lai, R., Li, K., Ren, Y., & Pan, J. Z. "An Extensive Evaluation of PDDL Capabilities in off-the-shelf LLMs." *arXiv preprint arXiv:2502.20175* (2025).

[58] Xi, Z., et al. "AgentPRM: Process Reward Models for LLM Agents via Step-Wise Promise and Progress." *arXiv preprint arXiv:2511.08325* (2025).

[59] Wang, H., et al. "Process-Supervised Reward Models for Verifying Clinical Note Generation: A Scalable Approach Guided by Domain Expertise." *arXiv preprint arXiv:2412.12583* (2024).

[60] Yang, X. W., Shao, J. J., Guo, L. Z., Zhang, B. W., Zhou, Z., Jia, L. H., Dai, W. Z., & Li, Y. F. "Neuro-Symbolic Artificial Intelligence: Towards Improving the Reasoning Abilities of Large Language Models." *arXiv preprint arXiv:2508.13678* (2025).